

Ray tracing – Curved quadratic elements

1. Tetrahedron and ray

Surface position on the tetrahedron's face (weighted sum of shape functions):

$$\mathbf{P}(g, h) = \sum_{i=0}^5 N_i(g, h) \cdot \mathbf{f}_{nodes_i} \quad (1)$$

Ray:

$$\mathbf{R}(t) = \mathbf{o} + t\mathbf{d} \quad (2)$$

where $\mathbf{d}$ is the ray's direction, and $\mathbf{o}$ is the ray's origin.

2. Intersection by minimising vector difference

2.1. Intersection problem

Face-ray intersection (3 unknowns) presented as the vector difference [1, 2]:

$$\mathbf{o} + t\mathbf{d} - \mathbf{P}(g, h) = \mathbf{0} \quad (3)$$

Nonlinear system of equations:

$$\mathbf{F}(t, g, h) = \mathbf{o} + t\mathbf{d} - \sum_{i=0}^5 N_i(g, h) \cdot \mathbf{f}_{nodes_i} = \mathbf{0} \quad (4)$$

The system of equations contains 3 unknowns and 3 equations, so the mapping is:

$$\Re^3 \rightarrow \Re^3 \quad (5)$$

They represent the distance vector between a point on the ray and a point on the surface.

2.2. Möller-Trumbore algorithm for linear triangle intersection (initial guess)

$$\mathbf{E}_1 = V_1 - V_0, \mathbf{E}_2 = V_2 - V_0 \quad (6)$$

$$\mathbf{p} = \mathbf{d} \times \mathbf{E}_2 \quad (7)$$

$$\det = \mathbf{E}_1 \cdot \mathbf{p} \quad (8)$$

$$\mathbf{t}_{vec} = \mathbf{o} - V_0 \quad (9)$$

Where V_0, V_1, V_2 are triangle vertices.

$$u = \frac{\mathbf{t}_{vec} \cdot \mathbf{p}}{\det} \quad (10)$$

$$\mathbf{q} = \mathbf{t}_{vec} \times \mathbf{E}_1 \quad (11)$$

$$v = \frac{\mathbf{d} \cdot \mathbf{q}}{\det} \quad (12)$$

$$t = \frac{\mathbf{E}_2 \cdot \mathbf{q}}{\det} \quad (13)$$

2.3. Newton-Raphson method

Jacobian of F :

$$\mathbf{J}_F = \left[\frac{\partial F}{\partial f}, \frac{\partial F}{\partial g}, \frac{\partial F}{\partial h} \right] \quad (14)$$

$$\mathbf{J}_F = \left[\mathbf{d}, -\frac{\partial P}{\partial g}, -\frac{\partial P}{\partial h} \right] \quad (15)$$

$$\frac{\partial \mathbf{P}}{\partial g} = \sum_{i=0}^5 \frac{\partial N_i}{\partial g} \cdot \mathbf{f}_{nodes_i} \quad (16)$$

$$\frac{\partial \mathbf{P}}{\partial h} = \sum_{i=0}^5 \frac{\partial N_i}{\partial h} \cdot \mathbf{f}_{nodes_i} \quad (17)$$

The update by solving a linear system:

$$\Delta = [\Delta t, \Delta g, \Delta h]^T \quad (18)$$

$$\mathbf{J}_F \cdot \Delta = -F(t, g, h) \quad (19)$$

$$\begin{bmatrix} \mathbf{d}_x & -\frac{\partial P_x}{\partial g} & -\frac{\partial P_x}{\partial h} \\ \mathbf{d}_y & -\frac{\partial P_y}{\partial g} & -\frac{\partial P_y}{\partial h} \\ \mathbf{d}_z & -\frac{\partial P_z}{\partial g} & -\frac{\partial P_z}{\partial h} \end{bmatrix} \begin{bmatrix} \Delta t \\ \Delta g \\ \Delta h \end{bmatrix} = -(\mathbf{o} + t\mathbf{d} - P(g, h)) \quad (20)$$

3. Intersection by minimising scalar squared distance

3.1. Intersection problem

Face-ray intersection could be presented as the scalar squared difference [3]:

$$F(g, h) = |\mathbf{V}|^2 - (\mathbf{V} \cdot \mathbf{d})^2 \quad (21)$$

where

$$\mathbf{V} = \mathbf{P}(g, h) - \mathbf{o} \quad (22)$$

and

$$t = \mathbf{V} \cdot \mathbf{d} = (\mathbf{P}(g, h) - \mathbf{o}) \cdot \mathbf{d} \quad (23)$$

The equation contains 2 unknowns, so the mapping is:

$$\Re^2 \rightarrow \Re^1 \quad (24)$$

They represent the squared perpendicular distance between the ray and surface (Figure 1).

If a local minimum of F is zero, then this corresponds to a point where the ray intersects the surface. Those points where F is a minimum, but where $F > 0$, indicate that the ray misses the surface by a finite distance. This is actually desirable in that the algorithm will still converge near "silhouette edges" of surface, but will give a non-zero minimum.

F could be split into 2 functions:

$$\mathbf{F}(g, h) = |\mathbf{V}|^2 - (\mathbf{V} \cdot \mathbf{d})^2 \quad (25)$$

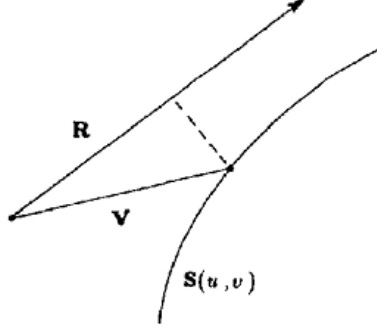


Figure 1: Distance from a point on the surface to a ray.

3.2. Quasi-Newton method

Jacobian of F :

$$\mathbf{J}_F = \left[\frac{\partial F}{\partial g}, \frac{\partial F}{\partial h} \right]^T \quad (26)$$

The partials of F are:

$$\frac{\partial F}{\partial g}(g, h) = 2 \left(\frac{\partial \mathbf{P}}{\partial g} \cdot \mathbf{V} - t \frac{\partial \mathbf{P}}{\partial g} \cdot \mathbf{d} \right) = \quad (27)$$

$$= 2 \frac{\partial \mathbf{P}}{\partial g} (\mathbf{V} - t\mathbf{d}) = \quad (28)$$

$$= 2 \frac{\partial \mathbf{P}}{\partial g} (\mathbf{P} - (\mathbf{o} + t\mathbf{d})) \quad (29)$$

$$\frac{\partial F}{\partial h}(g, h) = 2 \frac{\partial \mathbf{P}}{\partial h} (\mathbf{P} - (\mathbf{o} + t\mathbf{d})) \quad (30)$$

Hessian of F :

$$\mathbf{H}_F = \begin{bmatrix} \frac{\partial^2 F}{\partial g^2} & \frac{\partial^2 F}{\partial g \partial h} \\ \frac{\partial^2 F}{\partial h g} & \frac{\partial^2 F}{\partial h^2} \end{bmatrix} \quad (31)$$

where

$$\frac{\partial F}{\partial g^2}(g, h) = 2 \frac{\partial^2 \mathbf{P}}{\partial g^2} (\mathbf{P} + 1 - \mathbf{R}) \quad (32)$$

$$\frac{\partial F}{\partial h^2}(g, h) = 2 \frac{\partial^2 \mathbf{P}}{\partial h^2} (\mathbf{P} + 1 - \mathbf{R}) \quad (33)$$

$$\frac{\partial F}{\partial gh}(g, h) = \frac{\partial F}{\partial hg}(g, h) = 2 \frac{\partial^2 \mathbf{P}}{\partial gh} (\mathbf{P} + 1 - \mathbf{R}) \quad (34)$$

The update by solving a linear system:

$$\mathbf{\Delta} = [\Delta g, \Delta h]^T \quad (35)$$

$$\mathbf{H}_F \cdot \mathbf{\Delta} = -\mathbf{J}_F \quad (36)$$

$$\begin{bmatrix} \frac{\partial^2 F}{\partial g^2} & \frac{\partial^2 F}{\partial gh} \\ \frac{\partial^2 F}{\partial hg} & \frac{\partial^2 F}{\partial h^2} \end{bmatrix} \begin{bmatrix} \Delta g \\ \Delta h \end{bmatrix} = - \begin{bmatrix} \frac{\partial F}{\partial g} & \frac{\partial F}{\partial h} \end{bmatrix}^T \quad (37)$$

4. Surface normal

$$\mathbf{n} = \frac{\partial \mathbf{P}}{\partial g} \times \frac{\partial \mathbf{P}}{\partial h} \quad (38)$$

References

- [1] A. Glassner, An Introduction to Ray Tracing, The Morgan Kaufmann Series in Computer Graphics, Morgan Kaufmann, 1989. URL: <https://books.google.co.uk/books?id=t3XNCgAAQBAJ>.
- [2] D. L. Toth, On ray tracing parametric surfaces, SIGGRAPH Comput. Graph. 19 (1985) 171–179. URL: <https://doi.org/10.1145/325165.325233>. doi:10.1145/325165.325233.
- [3] K. I. Joy, M. N. Bhetanabhotla, Ray tracing parametric surface patches utilizing numerical techniques and ray coherence, SIGGRAPH Comput. Graph. 20 (1986) 279–285. URL: <https://doi.org/10.1145/15886.15917>. doi:10.1145/15886.15917.